

Physics of Diffusion Models in Generative AI

Name: Reneiloe Nyathela

Student Number: 2803374

Supervisor: Dr. Pallab Basu

Abstract

In this final version of my essay, I thoroughly glue everything I have covered since the start of this project. I explicitly cover and explain what diffusion models are as well as their properties. I cover the background and the mathematical tools used. I also go over generative AI and generative modelling. Since in this particular project I chose the more technical route, not the physics route, I built a neural network(NN) using PyTorch. The idea of the NN is to demonstrate how the predicted score-function can be used to remove noise from said-given data. The core idea was obtained from the DDPM famous paper. I work with real data to show this demonstration. Since this is still a physics project, I glue together how diffusion models, physics, and generative AI merge together. And also touch on the primary implications.

1 Introduction

Diffusion models in Machine Learning(ML) came out of ideas from non-equilibrium statistical Physics and score-based modelling. They then evolved into generative AI. These models are generative models used primarily for **image** generation, including other tasks in computer vision. Diffusion models are among the neural network architecture of generative modelling.[1]. Generative diffusion models are a family of generative models that slowly convert the score function or approximate lower bound of the data distribution to noise, then take the noise back in the reverse process to obtain a similar data distribution.[8] The diffusion model architecture represents a time conditioned denoising network that predicts noise, cleaning data given a noisy input x_t and a particular timestep t . The main backbones:

1.1 U-Net

Encoder-decoder with skip connection, for segmentation.

1.1.1 Encoder-Decoder

An **encoder** can be thought of as something that acts as a compressor by taking high dimensional data, like an image. This encoder extracts information[7] from the noisy image added. A **decoder**, which can be thought of as what uses the information from the encoder to rebuild the underlying structure of the noisy image. It uses the information extracted by the encoder to reconstruct the original image.

1.1.2 Skip Connections

Generally, while the encoder extracts the information from the noisy image, it is very most probable to lose a few or plethora of details about the noisy the image. From the image, those details could be, edges, small structures contained within the image, shapes and probably way more. So, skip connections ensures that the decoder does not miss those tiny details. This is crucial, considering the decoder's main objective is to produce the original image.[7, 6] So with missed information the original main image will not be correctly reconstructed. Thus, this process then takes takes information from earlier layers and gives it directly to a later decoder layer.

1.1.3 Segmentation

In short, segmentation means where and what each pixel belongs to. This statement emphasises that the model must know where exactly each and every detail/object is in the image. Correctly identifying their specific locations. Tying everything together, the encoder understands the big, underlying structure, the decoder's task is to reconstruct the image at pixel level and the skip connection helps the decoder remember the exact location and details within the image.

1.2 CNNs - Convolutional Neural Networks

CNN is a specific type of NN that For some low dimensional data, time series, robotics paths and some 3D tasks. CNNs are a type of NN that is good at recovering patterns that close to each other. Considering an image, a CNN observes properties like edges from a small group of pixels.[7] Instead of using CNNs to look at neighbouring in an image, we can us them to look at neighbouring values in sequences or paths, learning patterns between them.

1.3 Generative AI and Generative Modelling

Generative AI is a type AI that creates new content from learning patterns from existing data[9]. Generative modelling is in turn the ML framework. It is when a model creates data from noise. From diffusion models, GenAI learns how data changes when noise is

added, so it can then learn how to reverse the process. Beginning from noise and slowly retrieving structured data [10].

2 Theory

2.1 Principles of Diffusion Models

It is always best to think of diffusion models in the most simplest and straight forward way. Imagine you have hundreds of pictures of dogs. These pictures/images are an example of an unknown particular distribution. The distribution can be represented by $p_{data}(x)$, where x represents a specific image in the distribution. Up to now it is not known what the distribution looks like, this could be best imagined as every image being a point. The the regions where the images are found is where the data mostly concentrated. This is the sole reason why **generative modelling** is important, we need it to learn and understand the distribution to help us generate new samples of data from the already existing ones. While this method is effective, there are problems encountered, better laid out by Y. Hirono in: [11]. Hirono suggests that the two main problems, the first being we do not know the exact data distribution and must estimate the training set. The second being, despite knowing the exact distribution, sampling from it will be a great challenge. Taking from Hironos proposed hurdles, the only examples we have, given as:

$$x_1, x_2, x_3, \dots, x_N$$

The above are the training samples to learn, hence building a NN in assisting to do so. To combat Hironos first problem, assume a complicated unknown data distribution. We can then gradually add noise to it completely losing the underlying structure of the data, and then learn how to remove it. This helps in learning and understanding how the data behaves during these processes.

2.2 Formulation

This section of the essay highlights the mathematical tools used in the project. These tools are the basis of the project and are used in many areas of physics, machine learning and deep learning. The main connection of this project and physics is the Brownian motion, which is an old construct in particle and statistical physics that describes random motion of particles in fluids and Langevin dynamics.

2.2.1 Langevin dynamics and Brownian Motion

Long ago, Paul Langevin in an attempt to describe and understand motions of particles in molecular scales. When energy is considered, particles do not want to stay in a state with

high energy. So they try to go downhill and settle in low-energy states using the gradient of the energy surface [12]. He used these mathematical tools to describe the dynamics of particles, investigating their positions and paths, focusing on how they change over time in a potential energy $V(x)$:

$$dx = -\nabla_x V(x)dt + \sqrt{2}dB_t \quad (1)$$

A very simple way to this of the above equation intuitively is :

- dx - Describes the movement
- $-\nabla_x V(x)dt$ - Describes the downhill movement. This term points in the direction where the potential decreases, hence, the negative sign. Naturally going downhill.
- $\sqrt{2}dB_t$ - This term introduces randomness into the movement of the particle. Highlighting the fact that no matter the position of the particle, random collisions can bump it somewhere else.

Specifically dB_t is called the Brownian Motion and also described as the source of the noise[12]. The key connection to probability and potential energy can be expressed as $p_{data}(x) \propto e^{-V(x)}$. This relationship is very important as we will see after some manipulations that it leads to the score function, which is used during back propagation. The probability-potential energy relationship means the probability of finding a particle at some position x is related to its energy. To put into perspective, let us assume potentials at two different positions x_1 and x_2 , where x_1 being at the low energy state and x_2 being at the high energy state. Assuming the potentials are given as $V(x_1) = 1$ and $V(x_2) = 5$. The relationship and comparisons are, $p(x_1) \propto e^{-1}$ and $p(x_2) \propto e^{-5}$. So, it can be seen that $e^{-1} > e^{-5}$. This comparison supports the statement that particles are more likely to be found in a low energy state not a high energy state. The relationship is also part of an important score based model. It suggests negative energy can be treated as log likelihood:

$$p_{data}(x) \propto e^{-V(x)} \implies \log p_{data}(x)_{s(x)} = -V(x) + C \implies \underbrace{\nabla_x(x) \log p_{data}(x)}_{s(x)} = -\nabla_x V(x)$$

This leads directly to the score function. Substituting back to eq(1), then:

$$dx = s(x)dt + \sqrt{2}dB_t \quad (2)$$

2.2.2 Score

The score function is the gradient of the log likelihood function with respect to the model parameters. In maximum likelihood estimations, it provides information about the direction and magnitude on how the parameters should be adjusted to make the observed data

more probable. In diffusion models, the score function is the gradient of the logarithmic of the probability density of a variable, given by $s(x) = \nabla_x \log p_{data}(x)$ [12] where $p_{data}(x)$ is the probability density of the data after it has been corrupted by noise up to diffusion time t . Score based models train a NN to approximate this particular field. The forward diffusion process gradually destroys structure by adding noise:

$$dx = -\nabla_x V(x)dt + \sqrt{2}dB_t$$

At large t , $p_{data}(x)$ is usually close to a simple Gaussian distribution. The reverse process must move samples from this noise distribution back toward the data distribution. Its drift contains the score:

$$dx = [-\nabla_x V(x) - 2\nabla_x \log p_{data}(x)] dt + \sqrt{2}dB_t$$

with the reverse time interpretation of dt . Therefore, knowing the score tells the model how to correct a noisy state toward regions that are more likely under the data distribution. The reverse stochastic process uses this score to undo the forward diffusion.

A Gaussian example

Let a noisy data point be

$$x_t = x_0 + \sigma_t \epsilon \quad \epsilon \sim \mathcal{N}(0, I)$$

where

- σ_t is the noise coefficient, indicating how much Gaussian noise is added

Conditioned on the clean point x_0

$$p_{data}(x_t | x_0) = \mathcal{N}(x_0, \sigma_t^2 I)$$

The conditional score is

$$\nabla_{x_t} \log p_{data}(x_t | x_0) = \frac{x_t - x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}$$

So the score points from the noisy sample back toward its conditional mean, x_0 . In practice, the clean x_0 is unknown, so a NN learns to estimate the corresponding marginal score from noisy examples. This is the basis of denoising score matching, whose target is the analytically known conditional score:

$$\mathcal{L} = [\|s(x) - \nabla_{x_t} \log p_{data}(x_t | x_0)\|^2]$$

For Gaussian noise, the target is simply proportional to the negative noise, which is

why many diffusion implementations train a network to predict ϵ instead of explicitly predicting the score. So, the score function acts like a noise level dependent force that guides the reverse diffusion paths from random states toward physically configurations. [13]

2.2.3 Fokker-Planck Equation

The Fokker-Planck(FP) equation governs the time evolution of the probability density function (PDF) of a randomly evolving quantity, such as the position or velocity of a particle undergoing Brownian motion [14, 15, 18]. Because the underlying dynamics are stochastic, no single trajectory is predictable; instead, one studies the density obtained by averaging over many realizations of the noise,

$$P(x, t) = \langle \rho(x, t) \rangle \quad (3)$$

where $\rho(x, t)$ is the (singular) density associated with one noise realization and the angle brackets denote an ensemble average over noise. For a single stochastic variable x , the general one-dimensional FP equation reads

$$\frac{\partial}{\partial t} P(x, t) = -\frac{\partial}{\partial x} [A(x)P(x, t)] + \frac{1}{2} \frac{\partial^2}{\partial x^2} [B(x)P(x, t)] \quad (4)$$

where $A(x)$ and $B(x)$ are the drift and diffusion coefficients respectively, and both are required to be differentiable functions of x [18, 19]. Eq(3) is, in this special, position only form, also known as the **Smoluchowski equation** [18]. The two terms on the right hand side have distinct physical roles:

- $-\frac{\partial}{\partial x} A(x)P(x, t)$: the **drift term**, describing deterministic advection of probability
- $\frac{1}{2} \frac{\partial^2}{\partial x^2} B(x)P(x, t)$: the **diffusion term**, describing the spreading of probability due to random forcing. The transport term.

Identifying the correct forms of $A(x)$ and $B(x)$ for a given physical system is the main task in applying the FP equation and depends on the underlying stochastic process one starts from.

Derivation from the Langevin Equation Consider a Brownian particle of mass m moving in one dimension x with velocity v , subject to a viscous drag, a conservative force derived from a potential $V(x)$ and a fluctuating random force representing collisions with

the surrounding heat bath. The particle obeys

$$\frac{dx}{dt} = v \quad (5)$$

$$m \frac{dv}{dt} = -\gamma v + F(x) + \zeta(t) \quad (6)$$

where γ is the damping coefficient, friction and $\zeta(t)$ is a random (white-noise) force [16]. Writing the conservative force as $F(x) = -\partial V/\partial x$ and dividing Eq(6) by m gives the equivalent form

$$\frac{dv}{dt} = -\frac{\gamma}{m}v - \frac{1}{m}\frac{\partial V}{\partial x} + \frac{1}{m}\zeta(t) \quad (7)$$

The noise satisfies $\langle \zeta(t) \rangle = 0$, and its autocorrelation is a delta function in time (white noise),

$$\langle \zeta(t) \zeta(t') \rangle = 2\gamma k_B T \delta(t - t') \quad (8)$$

which is the statement of the fluctuation dissipation theorem relating the strength of the random force to the damping γ and the bath temperature T [20, 18].

Identifying the drift and diffusion coefficients Because the state of the system is now the pair (x, v) , the FP equation must be built in this 2D phase space, with one drift coefficient for each variable. From Eq. (5) the drift in position space is simply

$$A_x = v \quad (9)$$

and from the deterministic part of Eq(6) the drift in velocity space is

$$A_v = -\frac{\gamma}{m}v - \frac{1}{m}\frac{\partial V}{\partial x} \quad (10)$$

To obtain the diffusion coefficient, we write the noise using a standard Wiener process $W(t)$. The white noise is then expressed as

$$\zeta(t) = \sqrt{2\gamma k_B T} \eta(t) \quad (11)$$

where $\eta(t)$ is normalized Gaussian white noise related to the Wiener increment by

$$\eta(t) dt = dW \quad (12)$$

Substituting Eq(12) into Eq(11) gives the stochastic increment

$$\zeta(t) dt = \sqrt{2\gamma k_B T} dW \quad (13)$$

so that the change in velocity over an interval dt , from Eq. (7), is

$$dv = \left(-\frac{\gamma}{m}v - \frac{1}{m} \frac{\partial V}{\partial x} \right) dt + \frac{1}{m} \sqrt{2\gamma k_B T} dW \quad (14)$$

Comparing the noise term in Eq(14) with the general definition of the diffusion coefficient in an Itô stochastic differential equation [19], the velocity-space diffusion coefficient is identified as

$$B = \frac{2\gamma k_B T}{m^2} \quad (15)$$

The Fokker–Planck equation in phase space

Collecting the drift coefficients and the diffusion coefficient into the two variable form of Eq(3) gives

$$\frac{\partial P}{\partial t} = -\frac{\partial}{\partial x}(A_x P) - \frac{\partial}{\partial v}(A_v P) + \frac{1}{2} \frac{\partial^2}{\partial v^2}(B P) \quad (16)$$

Substituting Eqs(9), (10) and (15) into Eq(15) yields the Kramers form of the FP equation for a Brownian particle in a potential $V(x)$ [17, 18]:

$$\frac{\partial P}{\partial t} = -\frac{\partial}{\partial x}(v P) + \frac{\partial}{\partial v} \left[\left(\frac{\gamma}{m}v + \frac{1}{m} \frac{\partial V}{\partial x} \right) P \right] + \frac{\gamma k_B T}{m^2} \frac{\partial^2 P}{\partial v^2} \quad (17)$$

This is the fully derived FP equation. It is important to note that A_x depends on how many spatial dimensions are being considered, an additional pair of drift and diffusion terms of the same structure would appear for each further coordinate (e.g. y) if the motion were not restricted to one dimension [18].

Term by term interpretation

It is instructive to read Eq. (17) term by term.

Time evolution

$$\frac{\partial P}{\partial t} \quad (18)$$

represents the rate of change of the probability density at fixed (x, v) .

Drift in position space

$$-\frac{\partial}{\partial x}(v P) \quad (19)$$

is a continuity (transport) term. Since $v = dx/dt$, particles simply carry probability density with them as they move through position space; no probability is created or destroyed, only advected.

Drift in velocity space

$$\frac{\partial}{\partial v} \left[\left(\frac{\gamma}{m}v + \frac{1}{m} \frac{\partial V}{\partial x} \right) P \right] \quad (20)$$

represents deterministic changes in velocity, and splits into two physically distinct pieces:

- *Damping term*, $\frac{\gamma}{m}v$: viscous friction that causes the velocity to relax towards zero over time;
- *Force term*, $\frac{1}{m} \frac{\partial V}{\partial x}$: the conservative force $-\partial V/\partial x$ arising from the potential $V(x)$, which governs how external forces accelerate the particle.

Diffusion in velocity space

$$\frac{\gamma k_B T}{m^2} \frac{\partial^2 P}{\partial v^2} \quad (21)$$

represents diffusion due to thermal fluctuations, where $k_B T$ is the thermal energy of the bath. This term accounts for the random “kicks” the particle receives from its environment and is responsible for the spreading of the probability distribution in velocity space.

Reduction to the Diffusion Equation Returning to the general 1D FP equation, Eq(3), consider the special case in which the drift term vanishes

$$A(x) = 0 \quad (22)$$

Physically, this corresponds to a particle experiencing no systematic force, for instance, free Brownian motion in position space. Substituting Eq(21) into Eq(3), the drift term drops out and one is left with

$$\frac{\partial P}{\partial t} = \frac{1}{2} \frac{\partial^2}{\partial x^2} [B(x)P(x, t)] \quad (23)$$

If, in addition, the diffusion coefficient is treated as a constant, $B(x) = B$, it may be pulled outside the spatial derivative:

$$\frac{\partial P}{\partial t} = \frac{B}{2} \frac{\partial^2 P}{\partial x^2} \quad (24)$$

Defining the diffusion coefficient

$$D \equiv \frac{B}{2} \quad (25)$$

Eq(23) becomes

$$\frac{\partial P}{\partial t} = D \frac{\partial^2 P}{\partial x^2} \quad (26)$$

which is precisely the ordinary diffusion equation. Thus, in the force free, constant diffusion limit, the Fokker-Planck equation reduces exactly to the classical diffusion equation [18, 19].

3 Results and Discussion

This section of the essay will qualitatively demonstrate how the theory mentioned above. I show using real data how adding noise looks like. For specifically this project I used the Gaussian white noise(Normal distribution). Beginning with clean original images, described by the distribution:

$$x_0 \sim p_{data}(x) \quad (27)$$

3.1 Noise

Noise refers to the random signal that is gradually added to the data during the forward process and is then learned by the model to be removed during the reverse process. Gaussian white noise is most commonly used because it makes the mathematics simpler and the overall process easier to analyse. The idea is to move from $x_0 \rightarrow x_t$. The forward process is then defined by:

$$x_t = x_0 + \sigma_t \epsilon \quad \epsilon \sim \mathcal{N}(0, I) \quad (28)$$

The different noise levels used will be:

$$\sigma = [0.01, 0.05, 0.1, 0.2, 0.5, 1]$$

Where x_t is the corrupted image, x_0 is the original image. In this forward process, σ_t represents the amount noise added in the image in specific time intervals. And then $\epsilon \sim \mathcal{N}(0, I)$ is the normal distribution. The goal in this section is to properly build a NN that instead targets the the score function, $\hat{s}$. Initially the NN would be given information about the corrupted images and the noise level, then it gives out the score.

$$(x_t, \sigma_t) \rightarrow NN \rightarrow \hat{s}$$

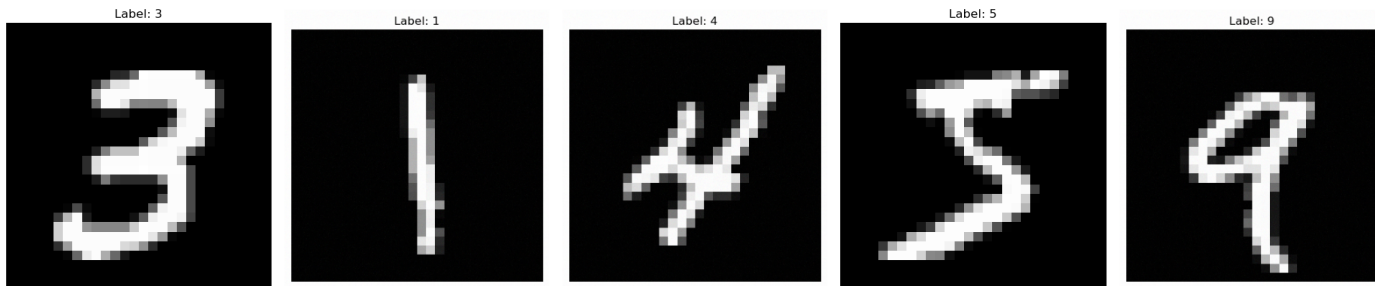
Conceptually, in our image example, the Gaussian white noise is introduced. We are interested in the different noise levels. Where the higher the noise levels, the more noise generated in the image. Although it is much advisable to use many different noise level, to track how much noise is being added in each timestep. This introduces the diffusion process. To track the noising process, we use values of t such that at $t = 0$ we have a clean, original image x_0 . Up to $t = T$, at which in this timestep we have lost most of the underlying details about the original image. So in essence, t can be thought of as a noise level inductor. Because the noise was added manually, we know exactly what the original image looks like, we also know all the different levels of the noise added as well as the shape of the images. The purpose of the NN would not to produce the original image, rather, we would ask it exactly which **direction** should we move towards in recovering

the original image. Following how the noise levels were introduced, we learn that this direction we are interested in is actually the **score**. The NN then learns the score as a descent approximation. Most of this section was completed through programming which I will include the repository of the project.

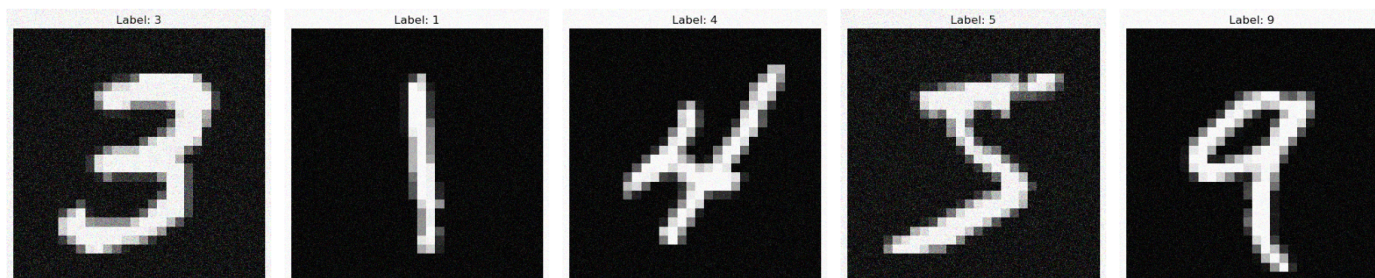


Scan to view the NN and files here

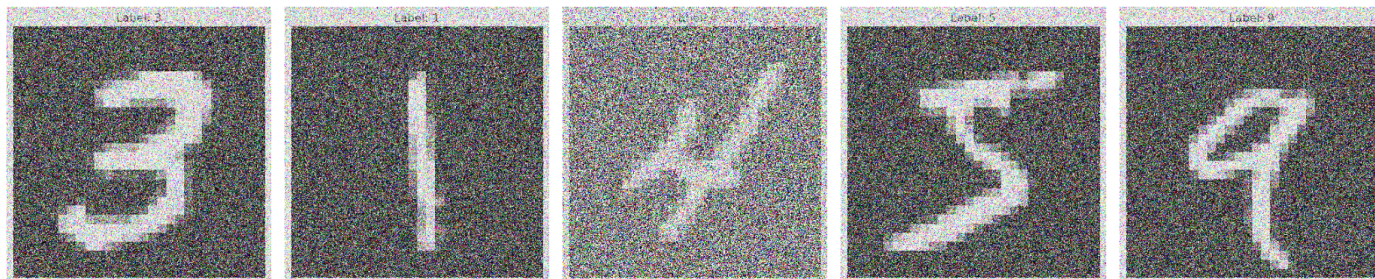
Below are the results from the forward process. The images below will show different intervals of t and also different values of the noise levels σ_t .



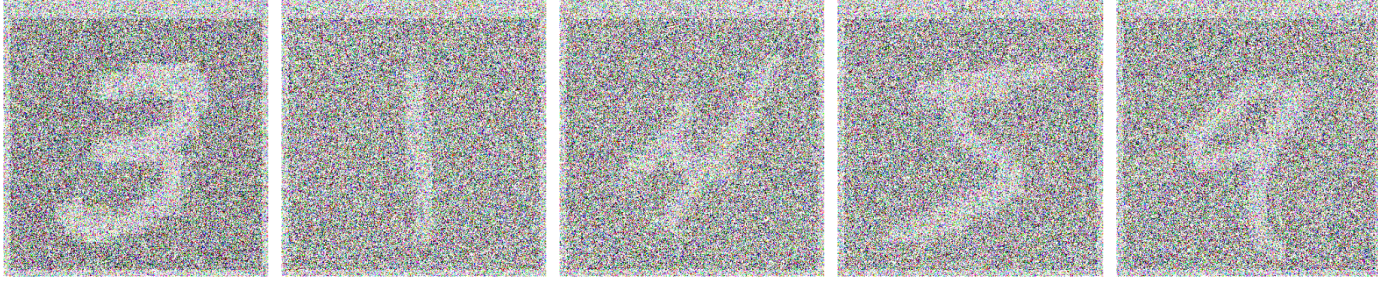
For $\sigma = 0$. The above are the original images, without noise.



For $\sigma = 0.05$.



For $\sigma = 0.1$



Skipping up to $\sigma = 1$

Table 1: Comparison of the measured noise and score standard deviations with their expected values for different values of σ .

σ	Noise std	Score std	Expected std
0.01	0.9997	99.9706	100.0000
0.05	1.0025	20.0504	20.0000
0.10	0.9997	9.9970	10.0000
0.20	0.9994	4.9969	5.0000
0.50	1.0006	2.0012	2.0000
1.00	1.0002	1.0002	1.0000

The noise was first added manually to the images using a Gaussian random distribution, where random values were generated and scaled by a chosen noise level σ . The noisy image was therefore created using $x_t = x_0 + \sigma\epsilon$, allowing us to control the amount of noise added. The corresponding target score was then calculated from the known noise using $\hat{s} = -\epsilon/\sigma$. The neural network was subsequently trained using these noisy images and their target scores. Rather than manually specifying the noise pattern, the NN learned the relationship between a noisy image, its noise level, and the direction required to remove the noise. During the reverse process, the NN receives the noisy image and predicts the score function. This predicted score is then used to gradually move the image toward a less noisy state. Repeating this process over multiple steps should produce the reconstructed image.

During training, the neural network was taught to predict the target score by comparing its predicted score, $s(x_t, \sigma)$, with the analytically calculated target score, $\hat{s} = -\epsilon/\sigma$. We used the mean squared error (MSE) as the loss function. The loss therefore measures how different the network’s predicted score is from the known target score. The network was trained over multiple epochs, where one epoch corresponds to one training iteration in our current setup. After calculating the loss, backpropagation was used to calculate how each network parameter contributed to the error. The gradients were then used by the Adam optimiser to update the parameters and reduce the loss. This process was repeated

for many epochs, allowing the network to gradually improve its score predictions. The resulting training loss plot can be placed here to show how the loss changed throughout training.

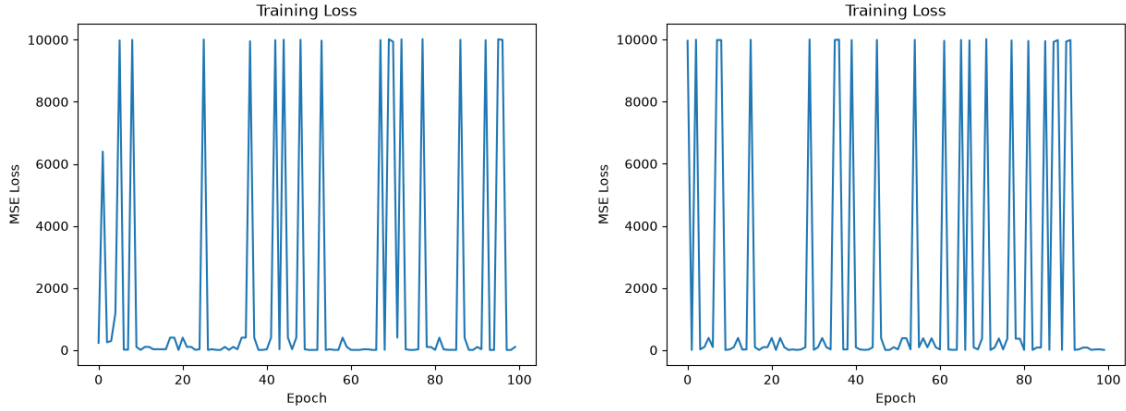


Figure 1: The above plots show how the error is behaving with different epochs

The peaks and drops are actually expected. The network is not necessarily getting worse when the graph spikes, the target itself is changing scale. And also it is worth noting that the network evaluates the images at random with random noise values. This does not mean the network is giving inaccurate results. We need to keep in mind that the target score, given by $\hat{s} = -(\epsilon/\sigma)$. For the first plot, the results given below from the computation:

Epoch	Noise level σ	Loss
0	0.05	229.5045
10	1.00	3.4655
20	0.05	399.3730
30	0.10	99.4500
40	0.20	24.2848
50	1.00	0.9281
60	0.50	3.2016
70	0.01	9939.0205
80	0.50	3.1684
90	0.10	97.9522

Table 2: Training loss at different epochs and noise levels σ .

The table below is for the second plot.

Epoch	Image	σ	Loss
0	Random	0.01	9958.0068
10	Random	0.20	21.4924
20	Random	0.05	390.4611
30	Random	1.00	7.1540
40	Random	0.10	90.2289
50	Random	0.20	23.0795
60	Random	0.20	20.7281
70	Random	0.05	372.5525
80	Random	0.50	5.3026
90	Random	0.01	9935.0059

Table 3: Training loss for randomly selected images at different noise levels and epochs.

The training loss plots show a relationship between the MSE through 100 epochs, the corresponding score and the noise level. The loss does not decrease smoothly because different values of σ were randomly selected during training. Since the target score is smaller than values of σ produce larger score magnitude. When $\sigma = 1$, the expected score standard deviation is a approximately 1, while for $\sigma = 0.1$, standard deviation increase to close to 10. Following that trend, the plot contains large upward peaks whenever a small value of σ is selected. This can be seen at epoch 0, where $\sigma = 0.01$ produces a loss of 9958.0068 and at epoch 90 where sigma is still the same. The loss is 9935.0059. In contrast, epochs using larger values produced much smaller losses, such as 7.1540 at epoch 30. The downward sections of the plot occur when larger noise levels are selected because their corresponding target scores have smaller magnitudes. At epoch 10, $\sigma = 0.2$ gave a loss of approximately 21.49, while at epoch 80, $\sigma = 0.5$ gave a loss of approximately 5.30. The results therefore show that the peaks are not necessarily failures of the NN but are largely a consequence of the $1/\sigma$ dependence of the score function. Across the 100 epochs, backpropagation and the Adam optimiser continually adjusted the network parameters to reduce the difference between the predicted and target scores. The decreasing losses observed for repeated noise levels, such as approximately $390 \rightarrow 372$ for $\sigma = 0.05$, indicate that the network was learning the score relationship. Thus, the up-and-down behaviour of the plot is primarily associated with changing σ , while the overall training process demonstrates that the network was learning to approximate the required score function.

After training, the learned network was used in the reverse process by taking a noisy image and predicting $s(x_t, \sigma)$. This predicted score was then used to update the image according to

$$x_{k+1} = x_k + \eta s(x_t, \sigma)$$

where η is the reverse-step size. By repeating this update, the image was gradually moved away from the noisy state towards a reconstructed image.

3.2 Reverse step

The reverse process uses the score, $\hat{s}$ to iteratively move the noisy image back towards the data distribution. After just a single reverse step, the first image noise removal is not very clear. It should take many steps to actually remove the noise completely. Below is the noise removal after one step. It shows the images from left to right, from the original, to noisy.

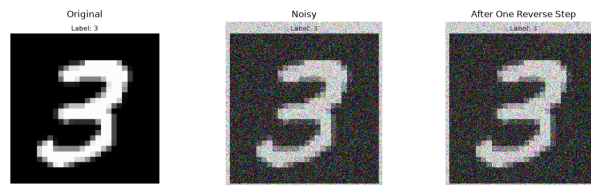


Figure 2: Image 3 evolution

The after 1000 reverse steps, we get something similar to the following.

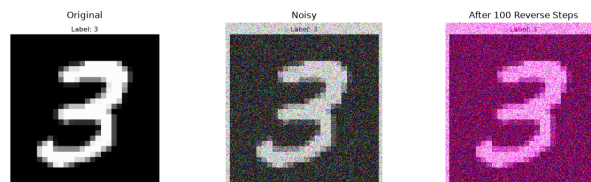


Figure 3: Image 3 evolution

What we are mostly interested in is that has the NN actually learned a useful score. The purple colour that appeared after 1000 reverse steps is mainly a result of accumulated error in the predicted score during the iterative reverse process. At every step, the neural network predicts the score separately for the red, green and blue channels. The reverse update is then applied repeatedly. Even if the predicted score is only slightly inaccurate, these errors accumulate over hundreds or thousands of steps. If the network consistently predicts slightly different corrections for the three colour channels, one channel can become larger than the others, producing a colour shift; in this case, the imbalance appeared as purple. We also kept $\sigma = 0.5$ fixed during the reverse iterations, whereas a proper diffusion reverse process gradually moves through a sequence of decreasing noise levels. Therefore, the same learned correction was repeatedly applied rather than progressively adapting to lower noise levels. The purple image therefore does not mean that the original

image contained purple information; it is an artefact of accumulated score-prediction and channel errors in our simplified reverse process.

4 Conclusion

This project explored the basic ideas behind score based diffusion models by bringing together Gaussian noise, NNs and the process of gradually removing noise. The images were first converted into numerical arrays so that they could be processed computationally in Python. Gaussian white noise was then added manually at different noise levels, σ , using the relation $x_t = x_0 + \sigma\epsilon$. Since the noise added to each image was known, the corresponding target score could be calculated using $\hat{s} = -\epsilon/\sigma$. This gave the NN a well defined mathematical quantity to learn. A CNN was then built and trained to predict the score from a noisy image while also taking the corresponding noise level into account.

The network was trained by comparing its predicted scores with the target scores using the (MSE). The Adam optimiser and backpropagation were then used to adjust the network's parameters over 100 epochs. Overall, the loss decreased during training, suggesting that the CNN was gradually learning how the noisy images relate to their corresponding score functions. The loss did not decrease smoothly and several large peaks appeared during training. This can be understood from the different σ values used. Since $\hat{s} \propto 1/\sigma$, smaller values of σ result in much larger target scores. These larger scores consequently lead to larger MSE values, which explains the ups and downs and peaks seen in the training loss.

Finally, the predicted score was used in an attempt to reverse the noise process and recover the original image. The image did not completely return to its original form and gradually developed a purple colour after several iterations. Although this was not the expected result, it provided a useful insight into the limitations of the simplified reverse process. Small errors in the predicted score can build up when the denoising step is repeated, especially when the same σ is used for every iteration. In a full diffusion model, the reverse process would instead follow a carefully chosen sequence of decreasing noise levels together with a more developed reverse SDE or sampling method. At the same time, the unsuccessful reconstruction highlighted some of the limitations of the simplified approach we used here. These results provide a useful starting point for improving the reverse process and moving towards a more complete diffusion

References

- [1] Yang, L., Zhang, Z., Song, Y., Hong, S., Xu, R., Zhao, Y., Zhang, W., Cui, B., Yang, M.-H., *Diffusion Models: A Comprehensive Survey of Methods and Applications*, ACM Computing Surveys, 2023. Available at: <https://arxiv.org/abs/2209.00796>
- [2] Croitoru, F., Hondru, V., Ionescu, R. T., Shah, M., *Diffusion Models in Vision: A Survey*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023. Available at: <https://www.computer.org/csdl/journal/tp/2023/09/10081412/1LRbIZMh0Uo>
- [3] Cao, H., Chen, C., Yang, Y., Shen, H., Wang, L., Yang, M.-H., et al., *A Survey on Generative Diffusion Models*, IEEE Transactions on Knowledge and Data Engineering, 2024. Available at: <https://www.computer.org/csdl/journal/tk/2024/07/10419041/1Udlmel27mg>
- [4] Shen, H., Chen, Y., Wang, J., Yang, L., Song, Y., et al., *Efficient Diffusion Models: A Survey*, Transactions on Machine Learning Research, 2025. Available at: <https://arxiv.org/abs/2502.06805>
- [5] Zhang, K., Song, J., et al., *Generative Diffusion Models: A Survey of Current Theoretical Developments*, Neurocomputing, 2024. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0925231224011445>
- [6] Zhiyuan Ma, Yuzhu Zhang, Guoli Jia, Liangling Zhao, Yichao Ma, Mingjie Ma, Gaofeng Liu, Kaiyan Zhang, Jianjun Li, Bown Zhou. , *Efficient Diffusion Models: A Comprehensive Survey from Principles to Practices*, arXiv:2410.11795v1 [cs.CV] 15 Oct 2024. Available at: <https://arxiv.org/pdf/2410.11795v1>
- [7] ROsa Wolf, Yitian Shi, Sheng Liu and Rania Rayyes , *Diffusion models for robotic manipulation: a survey* Front. Robot. AI, 09 September 2025. Available at: <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1606247/full>
- [8] Melike Nur Yeğın, Mehmet Fatih Amasyalı, *Generative diffusion models: A survey of current theoretical developments*, Science Direct 2024. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0925231224011445>
- [9] Kinza Yasar, textitWhat is a generative model?, Tech Accelerator What is GenAI? Generative AI explained 2025. Available at: <https://www.techtarget.com/ai/definition/What-is-a-generative-model>
- [10] Cole Stryker and Mark Scapicchio, *What is generative AI?*, IBM 2024. Available at : <https://www.ibm.com/think/topics/generative-ai>

- [11] Yuji Hirono, *Physics-Inspired Approaches in Generative Diffusion Models*, Journal of the Physical Society of Japan 94, 031008 (2025). Available at : <https://journals.jps.jp/doi/10.7566/JPSJ.94.031008>
- [12] Ayan Das, *Building Diffusion Model's theory from ground up*, May 7, 2024. Available at: <https://iclr-blogposts.github.io/2024/blog/diffusion-theory-from-scratch/>
- [13] Felix Koulischer, Johannes Deleu¹, Thomas Demeester¹, Gabriel Raya, Luca Ambrogioni, *DYNAMIC NEGATIVE GUIDANCE OF DIFFUSION MODELS*, Published as a conference paper at ICLR 2025. Available at: <https://arxiv.org/pdf/2410.14398v3>
- [14] A. D. Fokker, "Die mittlere Energie rotierender elektrischer Dipole im Strahlungsfeld," *Annalen der Physik*, vol. 348, no. 5, pp. 810–820, 1914.
- [15] M. Planck, "Über einen Satz der statistischen Dynamik und seine Erweiterung in der Quantentheorie," *Sitzungsberichte der Preussischen Akademie der Wissenschaften zu Berlin*, pp. 324–341, 1917.
- [16] P. Langevin, "Sur la théorie du mouvement brownien," *Comptes Rendus de l'Académie des Sciences*, vol. 146, pp. 530–533, 1908.
- [17] H. A. Kramers, "Brownian motion in a field of force and the diffusion model of chemical reactions," *Physica*, vol. 7, no. 4, pp. 284–304, 1940.
- [18] H. Risken, *The Fokker–Planck Equation: Methods of Solution and Applications*, 2nd ed., Springer Series in Synergetics. Berlin: Springer, 1996.
- [19] N. G. Van Kampen, *Stochastic Processes in Physics and Chemistry*, 3rd ed. Amsterdam: Elsevier, 2007.
- [20] R. Kubo, "The fluctuation–dissipation theorem," *Reports on Progress in Physics*, vol. 29, no. 1, pp. 255–284, 1966.
- [21] W. T. Coffey and Y. P. Kalmykov, *The Langevin Equation: With Applications to Stochastic Problems in Physics, Chemistry and Electrical Engineering*, 3rd ed. Singapore: World Scientific, 2012.
- [22] M. Elliott and E. Ginossar, "Applications of the Fokker–Planck equation in circuit quantum electrodynamics," *Physical Review A*, vol. 94, p. 043840, 2016. arXiv:1606.08508.

- [23] P. D. Drummond and D. F. Walls, “Quantum theory of optical bistability. I. Nonlinear polarisability model,” *Journal of Physics A: Mathematical and General*, vol. 13, no. 2, pp. 725–741, 1980.